

BC Experiment - 9

AIM: Implement the Private Ethereum Blockchain using Geth.

THEORY:

Introduction to Ethereum

Ethereum is an open-source, decentralized blockchain platform that goes beyond simple cryptocurrency transactions by supporting programmable smart contracts and decentralized applications (DApps). It was proposed by Vitalik Buterin in 2013 and launched in 2015. Unlike Bitcoin which is primarily a digital currency, Ethereum functions as a programmable distributed computing platform where developers can deploy self-executing code called smart contracts. The native cryptocurrency of the Ethereum network is called Ether (ETH), which is used to pay for computational operations on the network, referred to as gas fees.

Ethereum maintains a global state that is updated with every transaction and smart contract execution. Every node in the network holds a copy of this state, ensuring decentralization and fault tolerance. The network uses an account-based model where each account has a balance and can interact with other accounts or smart contracts.

What is a Private Ethereum Network?

A private Ethereum network is a standalone blockchain that replicates the Ethereum protocol but operates in complete isolation from the public mainnet. Organizations, developers, and academic institutions use private networks to test and develop blockchain applications without incurring real transaction costs or exposing data to the public chain. All network parameters such as consensus mechanism, block time, gas limits, and initial account balances are fully customizable. This makes private networks ideal for controlled experimentation, proof-of-concept development, and educational purposes.

What is Geth (Go-Ethereum)?

Geth, short for Go-Ethereum, is the most widely used official implementation of the Ethereum protocol, written in the Go programming language. It allows a computer to function as a full Ethereum node capable of mining Ether, validating transactions, executing smart contracts, and interacting with the blockchain. Geth can be operated through a command-line interface, a built-in JavaScript console, or via JSON-RPC API calls. It supports both public Ethereum networks as well as custom private networks, making it the preferred tool for setting up and managing private Ethereum deployments.

Consensus Mechanism — Clique (Proof of Authority)

Private Ethereum networks typically use the Clique consensus algorithm instead of Proof of Work. Clique is a Proof of Authority (PoA) protocol where only a set of pre-approved accounts called signers are authorized to produce and validate new blocks. Since block production does not involve solving computationally intensive puzzles, Clique is significantly faster and more energy efficient than traditional mining. The list of authorized signers is defined at the time of network creation and is embedded in the genesis block. This makes Clique well suited for private and consortium blockchain environments where participants are known and trusted.

Steps Involved in Implementing a Private Ethereum Network

1. Installing Geth: The first step involves downloading and installing the appropriate version of the Geth client on the system. Once installed, the binary is made accessible via the system PATH so it can be executed from any directory.

2. Creating Node Accounts: Each participating node requires an Ethereum account which consists of a public address and a corresponding private key stored in a keystore file. These accounts are created using the `geth account new` command and are protected by a password. The public addresses of these accounts are used in the genesis configuration.

3. Setting Up Password Files: To allow nodes to automatically unlock their accounts during startup without manual password entry each time, password files are created containing the account passwords in plain text.

4. Creating the Genesis Block: The genesis block is configured through a `genesis.json` file. This file defines the chain ID, consensus settings (Clique period and epoch), gas limit, initial Ether balances for each account, and the `extradata` field which encodes the authorized signer address. Every node in the network must be initialized with the same genesis file.

5. Initializing the Nodes: Each node's data directory is initialized using the `geth init` command along with the genesis file. This sets up the initial blockchain state and prepares the node to begin operating on the private network.

6. Running the Nodes: Node 1 is started as the signer node with mining enabled, responsible for sealing and producing new blocks. Node 2 is started as a member node that participates in the network but does not produce blocks. Both nodes are started with the Geth JavaScript console enabled for direct interaction.

7. Connecting the Nodes: Once both nodes are running, they are connected to each other using enode URLs. An enode is a unique identifier for a Geth node containing its public key and network address. Node 1's enode is retrieved using `admin.nodeInfo.enode` and then added to Node 2 using `admin.addPeer()`. Successful peering can be verified using `admin.peers`.

8. Sending Transactions: After the network is established and nodes are connected, Ether can be transferred between accounts using `eth.sendTransaction()` in the Geth console. The

transaction is picked up by the signer node, included in a block, and confirmed on the private chain. Account balances can be verified using `eth.getBalance()`.

PROCEDURE:

Step 1: Install Geth

`wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz`

`tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz`

`mkdir -p ~/bin`

`cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113`

`export PATH="$HOME/bin:$PATH"`

`geth113 version`

```
g2022_ansh_sarfare@cloudshell:~$ wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version
--2026-04-09 18:14:50-- https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 28248613 (27M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz'

geth-linux-amd64-1.13.14-2bd6bd01.tar.gz 100%[=====]

2026-04-09 18:14:59 (3.30 MB/s) - 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz' saved [28248613/28248613]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=/home/g2022_ansh_sarfare/gopath:google/gopath
```

Step 2: Create folders and accounts for both nodes

```
g2022_ansh_sarfare@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
[INFO] [04-09]15:36:17.038] Maximum peer count ETH=50 total=10
[INFO] [04-09]15:36:17.041] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x93b0c547de2cf8228981dd4a29ac3b0ca9a08859
Path of the secret key file: /home/g2022_ansh_sarfare/eth-private/node1/keystore/UTC--2026-04-09T15-36-35.166702641Z--93b0c547de2cf8228981dd4a29ac3d0ca9a08859

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

g2022_ansh_sarfare@cloudshell:~$

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
[INFO] [04-09]15:38:49.547] Maximum peer count ETH=50 total=50
[INFO] [04-09]15:38:49.548] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x6406586b898212f380a55b5f5e6596b095823d3b
Path of the secret key file: /home/g2022_ansh_sarfare/eth-private/node2/keystore/UTC--2026-04-09T15-38-54.774381074Z--6406586b898212f380a55b5f5e6596b095823b

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

Step 3: Create password files

```
g2022_ansh_sarfare@cloudshell:~$ echo "Ansh" > ~/eth-private/node1/password.txt
echo "Ansh" > ~/eth-private/node2/password.txt
g2022_ansh_sarfare@cloudshell:~$
```

Step 4: Create genesis.json and verify it looks correct

NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"

NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"

```
cat > ~/eth-private/genesis.json << EOF
```

[illegible]

[illegible]

Step 5: Initialize both nodes

Step 6: Run Node 1 (signer)

```
cloudshell x cloudshell x + v
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
2022_ansh_sarfare@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE1="93B0C547De2cF8228981Dd4a29aC3D0cA9a08859"
2022_ansh_sarfare@cloudshell:~$

2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
--networkid 1234 \
--port 30303 \
--http --http.port 8545 \
--unlock 0x5(NODE1) \
--password ~/eth-private/node1/password.txt \
--mine \
--minor-etherbase 0x5(NODE1) \
--allow-insecure-unlock \
console
INFO [04-09|15:52:04.688] Maximum peer count           ETH=50 total=50
INFO [04-09|15:52:04.691] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|15:52:04.694] Set global gas cap           cap=50,000,000
INFO [04-09|15:52:04.695] Initializing the KZG library  backend=gokzg
INFO [04-09|15:52:04.767] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [04-09|15:52:04.767] Using publie as the backing database database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata
INFO [04-09|15:52:04.767] Allocated cache and file handles database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-09|15:52:04.797] Opened ancient database      database=/home/2022_ansh_sarfare/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-09|15:52:04.798] State scheme out to already existing scheme=hash
INFO [04-09|15:52:04.799] Initialising Ethereum protocol network=1234 dbversion=<nil>
INFO [04-09|15:52:04.803]
INFO [04-09|15:52:04.803] -----
INFO [04-09|15:52:04.803] Chain ID: 1234 (unknown)
```

Step 7: Open new terminal tab and run Node 2 (member)

```
g2022_ansh_sarfare@cloudshell:11:~$ export PATH="$HOME/bin:$PATH"
NODE2="6406586b898212f380A55Bbf5e6596bD95825D3b"

g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node2 --networkid 1234 --port 30304 --http --http.port 8546 --unlock 0x5(NODE2) --
password ~/eth-private/node2/password.txt --allow-insecure-unlock console
INFO [04-09|16:10:33.188] Maximum peer count           ETH=50 total=50
INFO [04-09|16:10:33.188] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|16:10:33.193] Set global gas cap           cap=50,000,000
INFO [04-09|16:10:33.193] Initializing the KZG library  backend=gokzg
INFO [04-09|16:10:33.268] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [04-09|16:10:33.268] Using pebble on the backing database database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-09|16:10:33.268] Allocated cache and file handles database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-09|16:10:33.290] Opened ancient database      database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-09|16:10:33.300] State scheme set to already existing scheme=hash
INFO [04-09|16:10:33.301] Initialising Ethereum protocol network=1234 dbversion=8
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] -----
INFO [04-09|16:10:33.302] Chain ID: 1234 (unknown)
INFO [04-09|16:10:33.302] Consensus: Clique (proof-of-authority)
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] Pre-Merge hard forks (block based):
INFO [04-09|16:10:33.302] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-09|16:10:33.302] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-09|16:10:33.302] - Spurious Dragon/1 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon-1.md)
INFO [04-09|16:10:33.302] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon-2.md)
```

Step 8: Connect the two nodes

In Node 1 console, get its enode

```
> admin.nodeInfo.enode
"enode://a0f33306144ae52760abb42a72a912f3123189c228f201424f105b391186998f0fc78a77e454680d8898b9ca388ef647de953ae695bb21751e5f610baa5bb7d434.126.99.110:30303"
> INFO [04-09|16:22:47.013] Successfully sealed new block number=231 sealhash=bb4346..610566 hash=d1fc70..84fdc7 elapsed=5.001s
INFO [04-09|16:22:47.014] Commit new sealing work number=232 sealhash=8aca46..08c3ca txs=0 gas=0 fees=0 elapsed="538.827µs"
INFO [04-09|16:22:52.010] Successfully sealed new block number=232 sealhash=8aca46..08c3ca hash=d65637..b7846a elapsed=4.996s
INFO [04-09|16:22:52.010] Commit new sealing work number=233 sealhash=eldf14..1d6155 txs=0 gas=0 fees=0 elapsed="485.675µs"
INFO [04-09|16:22:52.270] Looking for peers peercount=0 tried=60 static=0
INFO [04-09|16:22:57.011] Successfully sealed new block number=233 sealhash=eldf14..1d6155 hash=0be857..1fcf8d elapsed=5.001s
INFO [04-09|16:22:57.012] Commit new sealing work number=234 sealhash=51ca87..5b8c95 txs=0 gas=0 fees=0 elapsed="501.891µs"
```

Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://acfe33406144ee52760abb42a72a912f3123188c228f201424f105b391186998f0fc78a77e454680d8898b9ca388cf647de953ae695bb21251e5f610baa5bb7d834.126.99.110:30303")
true
> [INFO [04-09|16:25:38.021] Looking for peers peercount=0 tried=45 static=1
```

In Node 2 console, check they are connected

```
> admin.peers
[{"caps": ["eth/68", "eth/69", "snap/1"],
  enode: "enode://00162513d2f3e10b7cdd0dc4a3d139842e3f9681147bf7367aa7fe40421ce3904f8a4ea0db1254cc2d0d8a0dfec08457c3938017bf424a952c92e70931183abb847.187.149.99:31404",
  id: "8d84e724deb8194376e99f1dd9493aeeed3cb404699b7695c94d147cbb8aeeb",
  name: "Nethermind/v1.36.0+31cb81b7/linux-x64/dotnet10.0.1",
  network: {
    inbound: false,
    localAddress: "10.88.0.4:55586",
    remoteAddress: "47.187.149.99:31404",
    static: false,
    trusted: false
  },
  protocols: {
    eth: "handshake",
    snap: "handshake"
  }
}]
```

Step 9: Send a transaction

In Node 1 console, send transaction to Node 2

```
> eth.sendTransaction({
  .... from: eth.accounts[0],
  .... to: "0x5fd63344c1f40712A9A85d9d7e2B304a6C558716",
  .... value: web3.toWei(1, "ether")
  .... })
[INFO [04-05|13:43:48.276] Setting new local account address=0x1D8b6d4f6c677D2fb66CA97FbD81bE2cd23c4D3
[INFO [04-05|13:43:48.276] Submitted transaction hash=0x566ac1b0f280ebd7b6d4a68991416a36248ed3ec43bd24f8a08af6badab79089 from=0x1D8b6d4f6c677D2fb66CA97FbD81bE2cd23c4D3 nonce=0 recipient=0x5fd63344c1f40712A9A85d9d7e2B304a6C558716 value=1,000,000,000,000,000
"0x566ac1b0f280ebd7b6d4a68991416a36248ed3ec43bd24f8a08af6badab79089"
```

and check the balance of Node 2

```
> eth.getBalance("0x5fd63344c1f40712A9A85d9d7e2B304a6C558716")
10100000000000000000
```

CONCLUSION

This experiment successfully demonstrated the implementation of a private Ethereum blockchain network using the Geth (Go-Ethereum) client. A two-node private network was set up from scratch, beginning with the installation of Geth, creation of individual node accounts, and configuration of the genesis block using the Clique Proof of Authority consensus mechanism. Both nodes were initialized with the same genesis file, started with appropriate network parameters, and successfully connected to each other using enode URLs.

Once the network was operational, the signer node (Node 1) began producing and sealing blocks at regular intervals, confirming that the consensus mechanism was functioning correctly. The two nodes were peered together and their connection was verified using the `admin.peers` command. A transaction was then executed from Node 1 to Node 2 using `eth.sendTransaction()`, and the updated balance of Node 2 was confirmed using `eth.getBalance()`, validating that transactions are being processed and recorded on the private chain.

Through this experiment, a practical understanding of how Ethereum nodes operate, how private blockchain networks are configured, and how peer-to-peer communication works

between nodes was gained. It also highlighted the advantages of using Proof of Authority consensus in private networks, particularly in terms of faster block production and reduced computational overhead compared to Proof of Work. Overall, this experiment provided hands-on exposure to core blockchain concepts including genesis block configuration, node initialization, peer discovery, and on-chain transaction verification.